Evaluating Snapshot-Consistent Live Migration for Stateful Containers in Kubernetes: A CRIU and Sidecar-Free eBPF Case Study

Musa Khasmammadov
Department of Computer Engineering & Cloud Systems
Baku, Azerbaijan
musaxasmammedov77@gmail.com

Abstract—While stateless microservices in Kubernetes achieve high availability through multi-replica deployments, rolling updates, and PodDisruptionBudgets, stateful singleton workloads lacking built-in persistence—such as legacy scientific simulations, in-memory session aggregators, and custom stateful inference caches—cannot be trivially replicated without distributed consensus overhead or intrusive application refactoring. When cluster nodes undergo maintenance or ephemeral cloud infrastructure (e.g., Amazon Web Services EC2 Spot instances) is reclaimed via a two-minute preemption notification, the standard Kubernetes eviction lifecycle (SIGTERM followed by SIGKILL) destroys all volatile in-memory heap state.

This paper designs, implements, and evaluates a snapshot-consistent live migration architecture for stateful Kubernetes workloads. The pipeline coordinates the Linux Checkpoint/Restore in Userspace subsystem (CRIU v4.3) via the Kubelet Checkpoint API (KEP-2008), OCI artifact packaging via Buildah, the CRI-O container runtime (v1.31), and a sidecar-free Cilium eBPF networking stack. We formalize and measure the trade-off between client-observed cutover blackout (Recovery Time Objective, RTO) and snapshot state drift (Recovery Point Objective, RPO).

Across $N = 36$ live migrations executed between AWS EC2 worker nodes, the decoupled pipeline achieves a median cutover blackout of 1064.5 ms (minimum 797 ms, mean 1132.4 ± 241.1 ms, $p95 = 1532$ ms) and a median process freeze time of 144.0 ms (mean 144.2 ± 8.4 ms), yielding a median cumulative disruption of 1203.0 ms (minimum 944 ms) compared to 1289 ms for a cold reschedule baseline that loses 100% of RAM state. Linear regression across all 36 runs ($r = 0.9624$) reveals that CRI-O unfreezes the restored process at $\approx$ 537 ms, while the remaining blackout variance stems from Kubelet's 1 s Pod Lifecycle Event Generator (PLEG) status synchronization loop. Source-side state drift is tightly bounded to 7.5 ± 0.6 ticks ($\approx$ 669 ms RPO at 10 Hz). Finally, we formulate a self-consistent analytical scaling model for the source-side 120-second Spot evacuation deadline and release all artifacts as open-source.

Index Terms—Kubernetes, Container Live Migration, CRIU, CRI-O, Cilium eBPF, Stateful Workloads, Spot Instances, Recovery Point Objective.

I. Introduction

Container orchestration platforms, led by Kubernetes, are built around microservice idempotency and disposable instances [1]. In stateless deployments, node drain operations (kubectl drain) and instance preemptions are handled transparently by provisioning replacement pods prior to terminating old replicas via rolling update parameters (maxSurge, PodDisruptionBudget).

However, a distinct class of cloud workloads relies on long-running volatile memory state without built-in distributed checkpointing:

1) Legacy and Scientific Single-Process Runtimes: Numerical solvers, physics simulations, and monolithic C++/Python applications that accumulate gigabytes of heap state in RAM without native write-ahead logging.
2) In-Memory Session Aggregators and Caches: Stateful middleware and warm inference caches where reconstructing in-memory data structures from cold storage incurs prohibitive warm-up latency.

When these stateful singleton workloads run on ephemeral cloud instances—most notably Amazon Web Services (AWS) EC2 Spot instances, which offer 70–90% cost reductions relative to On-Demand pricing but are subject to preemption with a strict 120-second interruption notice (often preceded by an early EC2 Rebalance Recommendation signal) [2], [3]—standard eviction causes complete state loss. When a node is drained, the kubelet issues SIGTERM; upon expiration of terminationGracePeriodSeconds (or hard Spot preemption), SIGKILL destroys the process address space.

A common pattern to avoid state loss is externalizing volatile variables to an in-memory data store such as Redis [4]. While effective for shared state across replicas, externalizing internal process memory requires intrusive source-code refactoring and replaces nanosecond-scale local RAM accesses with sub-millisecond network round-trips across nodes.

Container checkpoint/restore offers an application-transparent alternative [5]–[7]. In this paper, we evaluate an end-to-end live migration pipeline built on native Kubernetes primitives. Our contributions are:

- Decoupled Migration Pipeline: We implement an automated DaemonSet operator combining KEP-2008 checkpointing [8], Buildah OCI packaging, CRI-O v1.31 restoration, and Cilium eBPF routing across AWS EC2 nodes without modifying application code.

- Empirical Statistical Characterization ($N = 36$): We evaluate 36 back-and-forth live migrations, decomposing client-perceived disruption into source-side freeze latency (T_{freeze}), target-side container restoration ($T_{\text{unfreeze}} \approx 537$ ms), and Kubelet PLEG status propagation delay (Δt_{pleg}).
- RTO vs. RPO Trade-off and Time-Rollback Analysis: We formalize both decoupled (KeepRunning: true) and synchronous stop-and-copy modes, explaining the relationship between source-side state drift (ΔS_{lost}) and post-restore target execution ($\Delta S_{\text{post-restore}}$).
- Source-Side Spot Evacuation Scaling Model: We derive an affine latency scaling model calibrated to empirical fixed overheads and evaluate source-side evacuation feasibility under the 120-second Spot deadline for heaps from 26 MB to 8 GB.

II. System Architecture

A. Pipeline Overview and CRIU Configuration

The migration architecture integrates five components on Ubuntu 24.04 LTS (Linux kernel 6.8): CRIU v4.3 [7], the Kubelet Checkpoint API (KEP-2008) [8], the CRI-O runtime (v1.31) configured with the runc OCI runtime, Buildah for OCI image packaging, and Cilium 1.16 in kubeProxyReplacement mode.

To allow a checkpointed container to restore inside a new network namespace with a different Pod IP on the target node, /etc/criu/default.conf is configured with five directives [7]:

- tcp-close: Restores listening sockets (0.0.0.0:8090) in the LISTEN state while closing any established sockets that are bound to the old Pod IP.
- skip-in-flight: Instructs CRIU to ignore transient in-flight TCP connection handshakes occurring during active 50 Hz probing at the moment of ptrace seizure.
- skip-mnt /run/.containerenv: Excludes CRI-O's transient metadata file mount, preventing mountpoint validation failures during CRImportCheckpoint.
- ext-unix-sk and shell-job: Permits serialization of external UNIX domain sockets and container stdio descriptors.

B. Network Cutover and Blackout Definition

When a pod migrates from k8s-worker-1 (10.244.2.0/24) to k8s-worker-2 (10.244.1.0/24), its Pod IP changes. Our test workload serves HTTP/1.0 responses (closing the TCP connection after each reply) behind a Kubernetes Service ClusterIP and Cilium L7 Envoy Ingress without long-hold request buffering.

Consequently, during the cutover interval between source pod deletion (t_{kill}) and target endpoint readiness (t_{ready}), probes arriving at the Service or Ingress fail immediately (connection refused / HTTP 503). We define the cutover blackout window T_{blackout} strictly as the unavailability interval during which consecutive probes fail—measured both internally by the operator clock ($t_{\text{ready}} - t_{\text{kill}}$) and externally as the maximum time gap between consecutive HTTP 200 OK responses on a 50 Hz (±20 ms) prober.

C. Rationale for Sidecar-Free Networking

Kubelet's Checkpoint API (POST /checkpoint/{ns}/{pod}/{container}) checkpoints an individual container rather than an entire pod [8]. In a traditional sidecar service mesh (e.g., Istio), the istio-proxy Envoy container runs as a separate container sharing the pod's network namespace (netns) and intercepting traffic via pod-local iptables rules and 127.0.0.1 loopback connections.

We hypothesize and argue from an architectural standpoint that sidecar-free eBPF networking (Cilium) simplifies container checkpointing for two reasons: (i) checkpointing only the application container leaves behind the sidecar's iptables NAT state and any persistent application-to-sidecar loopback sockets; and (ii) Cilium places L7 Envoy proxying and eBPF socket routing at the host level outside the pod namespace, keeping the container's netns clean. Confirming this experimentally against an Istio sidecar baseline and evaluating persistent HTTP/1.1 keep-alive connections remain part of our future work.

III. Mathematical Latency, Drift, and Scalability Model

A. Latency Decomposition and Timestamp Semantics

In our operator implementation (05-live-migration-operator.yaml), timestamps are recorded sequentially around each stage: t_0 (before checkpoint API call), t_1 (after checkpoint tar creation), t_2 (after Buildah commit), t_3 (after registry push), t_{kill} (after the pre-delete counter probe c_{kill}, immediately before DELETE /pods), and t_4 (upon the first HTTP 200 OK from the restored pod after Kubelet reports phase == Running).

The stage durations are therefore:

$$T_{\text{freeze}} = t_1 - t_0, \quad T_{\text{build}} = t_2 - t_1, \quad T_{\text{push}} = t_3 - t_2 \tag{1}$$

$$T_{\text{restore}} = t_4 - t_3, \quad T_{\text{blackout}} = t_4 - t_{\text{kill}} = T_{\text{restore}} - t_{\text{probe}} \tag{2}$$

where $t_{\text{probe}} = t_{\text{kill}} - t_3 \approx 1$–$2$ ms is the latency of the pre-delete HTTP probe used to read c_{kill}. This explains why operator-logged T_{blackout} is consistently 1–2 ms shorter than T_{restore} (e.g., 797 ms vs. 798 ms, and 1078 ms vs. 1080 ms).

Total client-perceived disruption combines the source-side freeze stutter (when threads are halted by ptrace and incoming requests stall in kernel socket queues) and the cutover blackout:

$$T_{\text{disruption}} = T_{\text{freeze}} + T_{\text{blackout}} \tag{3}$$

while total end-to-end orchestration duration is:

$$T_{\text{total}} = T_{\text{freeze}} + T_{\text{build}} + T_{\text{push}} + T_{\text{restore}} \tag{4}$$

The 9-Stage Stateful Container Migration Sequence:
1. Trigger: Manual API invocation, node maintenance label, or AWS Spot Interruption / Rebalance signal (T_{detect}).
2. Pre-Dump Probe & Memory Freeze: Operator records pre-dump counter c_{dump} and calls POST :10250/checkpoint/boutique/pod/container. CRIU halts threads via ptrace(PTRACE_SEIZE), serializes anonymous pages to checkpoint.tar, and unfreezes the source container (T_{freeze}, median 144.0 ms).
3. OCI Packaging: Buildah packages checkpoint.tar into a gzip-compressed OCI layer ($\approx$ 5.16 MB) annotated with io.kubernetes.cri-o.annotations.checkpoint.name (T_{build}, median 370.5 ms).
4. Registry Push: The OCI checkpoint image is pushed to the cluster-internal registry at 10.10.1.10:5000 (T_{push}, median 298.5 ms). Source-side evacuation is now complete.
5. Pre-Kill Probe & Cutover Initiation: Operator samples c_{kill} (t_{kill}) and deletes the source pod with gracePeriodSeconds=0; the cutover blackout window (T_{blackout}) begins.
6. Target Scheduling: Target Pod manifest referencing the checkpoint OCI image is created with spec.nodeName pinned to the destination worker.
7. CRI-O Pull & Restore: Target CRI-O pulls the checkpoint layer, validates matching bind mounts (automountServiceAccountToken: false), and invokes runc/criu restore, unfreezing the process at $T_{\text{unfreeze}} \approx$ 537 ms.
8. Process Resumption: Target process re-binds 0.0.0.0:8090 and immediately resumes its 10 Hz counter loop in RAM.
9. PLEG Sync & Readiness Verification: Kubelet updates Pod status.phase to Running in kube-apiserver; operator verifies HTTP 200 OK from the new Pod IP and records c_{restore} (T_{restore}, median 1066.0 ms).

Fig. 1. Architectural sequence of the decoupled snapshot-consistent live migration pipeline.

B. Decomposing Target Restoration and Counter Semantics

The operator logs three counter samples per migration: Counter($c_{\text{dump}} \to c_{\text{kill}} \to c_{\text{restore}}$), where:

- c_{dump} is sampled on the source pod immediately prior to t_0.
- c_{kill} is sampled on the source pod at t_{kill} (after $T_{\text{freeze}}+T_{\text{build}}+T_{\text{push}}$).
- c_{restore} is sampled on the target pod at t_4 (after Kubelet transitions the target pod to status.phase == Running).

Because CRI-O invokes CRIU with KeepRunning: true, the source process continues incrementing at rate $\lambda =$ 10 Hz during $T_{\text{build}} + T_{\text{push}}$ (and during the brief pre-seizure setup inside T_{freeze}). If the checkpoint captures state $S_{\text{ckpt}} \approx c_{\text{dump}}+1$, the true number of source updates lost at cutover is:

$$\Delta S_{\text{lost}} = c_{\text{kill}} - S_{\text{ckpt}} \approx \lambda \cdot (T_{\text{build}} + T_{\text{push}}) \tag{5}$$

Correspondingly, the Recovery Point Objective (RPO) is:

$$\text{RPO} = \Delta t_{\text{drift}} = T_{\text{build}} + T_{\text{push}} \tag{6}$$

Crucially, why is c_{restore} strictly greater than c_{dump} (e.g., $51 \to 60 \to 57$ in Run 1 and $77 \to 85 \to 79$ in Run 2)? On the target node, CRI-O and runc restore unfreeze the container at time T_{unfreeze} after t_3, at which point the restored Python thread immediately resumes incrementing from S_{ckpt} at 10 Hz. However, the operator cannot probe the target pod IP until Kubelet's Pod Lifecycle Event Generator (PLEG, which relists container states on a 1 s periodic cycle) updates Pod.status.phase = Running in kube-apiserver at time $T_{\text{restore}} = T_{\text{unfreeze}} + \Delta t_{\text{pleg}}$. During Δt_{pleg}, the restored process accumulates $\Delta S_{\text{post-restore}}$ new ticks on the target node:

$$c_{\text{restore}} - c_{\text{dump}} \approx \lambda \cdot (T_{\text{restore}} - T_{\text{unfreeze}}) \tag{7}$$

Consequently, the difference $c_{\text{kill}} - c_{\text{restore}}$ (3 in Run 1; 6 in Run 2) is not the number of lost updates ΔS_{lost}, but rather $\Delta S_{\text{lost}} - \Delta S_{\text{post-restore}}$.

C. RTO vs. RPO: Decoupled vs. Stop-and-Copy

In Mode A (Decoupled, KeepRunning: true), the source serves traffic during $T_{\text{build}} + T_{\text{push}}$, yielding RPO $= T_{\text{build}} + T_{\text{push}}$ ($\approx$ 669 ms median) and $T_{\text{blackout}} \approx T_{\text{restore}}$. Clients reading state between t_1 and t_{kill} observe counter values up to c_{kill} that roll back to S_{ckpt} when the target starts; transactional workloads therefore require an external write-ahead log (WAL) or idempotency keys.

In Mode B (Synchronous Stop-and-Copy, Analytical Reference), stopping the source container prior to dumping eliminates state drift (RPO $\equiv$ 0, $\Delta S_{\text{lost}} \equiv 0$), but extends service unavailability to the entire pipeline ($T_{\text{blackout}}^{\text{stop-and-copy}} = T_{\text{total}}$, median 1878.0 ms, or 45.7% slower than cold reschedule).

D. Source-Side Scalability Model for AWS Spot Preemption

When an AWS EC2 Spot instance receives a 120-second interruption notice, only the source node is terminated at $t = 120$ s. Once the checkpoint image is pushed to the registry (t_3), source evacuation is complete; target-side pull and restoration (T_{restore}) proceed on the surviving

destination node. Thus, the Spot deadline applies strictly to the source-side evacuation time $T_{\text{evac}}(M)$:

$$T_{\text{evac}}(M) = T_{\text{detect}} + T_{\text{freeze}}(M) + T_{\text{build}}(M) + T_{\text{push}}(M) < 120 \text{ s} \quad (8)$$

where $T_{\text{detect}} = 2.50$ s is the mean detection latency for a 5 s IMDSv2 poll interval. We model each stage as an affine function of dumpable heap size M (in MB), separating fixed orchestration overheads from bandwidth-bound transfer:

$$T_{\text{freeze}}(M) = t_{\text{dump},0} + \frac{M}{B_{\text{IO}}}, \quad T_{\text{build}}(M) = t_{\text{build},0} + \frac{M}{B_{\text{tar}}} \quad (9)$$

$$T_{\text{push}}(M) = t_{\text{push},0} + \frac{M}{B_{\text{net}}} \quad (10)$$

$$\text{RPO}(M) = T_{\text{build}}(M) + T_{\text{push}}(M) \quad (11)$$

$$T_{\text{restore}}(M) = t_{\text{restore},0} + \frac{M}{B_{\text{net}}} + \frac{M}{B_{\text{restore}}} \quad (12)$$

$$T_{\text{total}}(M) = T_{\text{freeze}}(M) + \text{RPO}(M) + T_{\text{restore}}(M) \quad (13)$$

Note that $T_{\text{restore}}(M)$ explicitly includes both pulling the image over the network (M/B_{net}) and unpacking/mapping pages on the target node (M/B_{restore}). Calibrating fixed overheads to our $N = 36$ empirical medians at $M = 26$ MB gives $t_{\text{dump},0} = 0.04$ s, $t_{\text{build},0} = 0.24$ s, $t_{\text{push},0} = 0.19$ s, and $t_{\text{restore},0} = 0.79$ s, with conservative sustained uncached bandwidths $B_{\text{IO}} = 125$ MB/s (EBS gp3), $B_{\text{tar}} = 200$ MB/s, $B_{\text{net}} = 250$ MB/s (2 Gbps sustained on m7i-flex.large), and $B_{\text{restore}} = 150$ MB/s. Every entry in Table I is computed directly from Eqs. (8)–(13).

TABLE I
Analytical Scaling Projection from Eqs. (8)–(13) vs. 120s Source-Side Spot Deadline

Heap M	T_{freeze}	T_{build}	T_{push}	RPO	T_{restore}	T_{evac} (% SLA)	T_{total}
26 MB (Model)*	0.25 s	0.37 s	0.29 s	0.66 s	1.07 s	3.41 s (2.8%)	1.98 s
256 MB	2.09 s	1.52 s	1.21 s	2.73 s	3.52 s	7.32 s (6.1%)	8.34 s
1024 MB (1 GB)	8.23 s	5.36 s	4.29 s	9.65 s	11.71 s	20.38 s (17.0%)	29.59 s
4096 MB (4 GB)	32.81 s	20.72 s	16.57 s	37.29 s	44.48 s	72.60 s (60.5%)	114.58 s
8192 MB (8 GB)	65.58 s	41.20 s	32.96 s	74.16 s	88.17 s	142.23 s (118.5%)	227.91 s

*At $M = 26$ MB, measured median T_{freeze} is 0.14 s (vs. 0.25 s uncached model) because Linux page cache buffers the 5.16 MB compressed checkpoint archive in RAM.

Table I shows two distinct regimes: (i) source-side Spot evacuation ($T_{\text{evac}} < 120$ s) succeeds for heaps up to 4 GB (72.60 s, or 60.5% of the 120 s budget) but fails at 8 GB (142.23 s); and (ii) even at 1 GB, single-pass dumping freezes the process for $T_{\text{freeze}} = 8.23$ s and incurs RPO = 9.65 s, confirming that multi-gigabyte workloads require iterative pre-copying.

IV. Empirical Evaluation

A. Experimental Setup

The testbed was deployed on AWS EC2 in eu-central-1 (Frankfurt):

- Nodes: 1x c7i-flex.large control plane (10.10.1.10, hosting the local OCI registry :5000 and Cilium Ingress) and 2x m7i-flex.large workers (k8s-worker-1, k8s-worker-2).
- Background Contention: All 11 microservices of the Google Online Boutique suite were deployed across the worker nodes to create realistic multi-tenant CPU, memory, and CNI control-plane contention.
- Target Workload: live-stream-pod running an in-memory 10 Hz counter on port 8090 (python:3.12-slim pre-pulled on both workers).

B. Statistical Results Across $N = 36$ Migrations

Table II reports the complete statistical distribution across all $N = 36$ live migrations recorded in cluster etcd ($n = 20$ from worker-1 to worker-2; $n = 16$ from worker-2 to worker-1), and Table III compares two representative individual runs against the native cold reschedule baseline and analytical Mode B.

C. Analysis of Findings

1) Source-Side Determinism vs. Target-Side PLEG Variance: Across all $N = 36$ runs, source-side stages exhibit tight determinism: $T_{\text{freeze}} = 144.2 \pm 8.4$ ms, $T_{\text{build}} = 369.0 \pm 16.7$ ms, and $T_{\text{push}} = 294.9 \pm 12.4$ ms. In contrast, T_{restore} spans 798–1688 ms (SD = 241.1 ms; range 890 ms).
2) Proof of Unfreeze Timing via Counter Regression: Performing ordinary least-squares regression of $(c_{\text{restore}} - c_{\text{dump}})$ against T_{restore} across all $N = 36$ runs confirms Eq. (7) with Pearson correlation $r = 0.9624$, yielding slope $\hat{\lambda} = 9.96$ ticks/s (matching the 10.0 Hz timer) and x-intercept $T_{\text{unfreeze}} = 537.1$ ms. Thus, CRI-O and runc restore actually unfreeze the container on the target node in ≈ 537 ms; the remaining 261–1151 ms of T_{restore} is Kubelet's 1 s PLEG status sync delay before reporting phase == Running.
3) Resolution of the Run 1 vs. Run 2 Counter Paradox: Across all $N = 36$ runs, $c_{\text{kill}} - c_{\text{dump}}$ is tightly bounded between 7 and 9 ticks (median 7.0, mean 7.5 ± 0.6), matching Eq. (5). In Trace #35 ($51 \to 60 \to 57$, $T_{\text{restore}} = 1080$ ms), the source incremented 9 times ($51 \to 60$), and after unfreezing at ≈ 537 ms on the target node, the restored process ticked ≈ 5 times during the 543 ms PLEG wait ($52 \to 57$). In Trace #36 ($77 \to 85 \to 79$, $T_{\text{restore}} = 798$ ms), the PLEG wait was only 261 ms ($798 - 537$ ms), so the restored counter ticked only $\approx$ 1–2 times ($78 \to 79$).

V. Related Work

Clark et al. [10] introduced iterative pre-copy live migration for virtual machines. In container environments, Stoyanov and Kollingbaum [5] evaluated CRIU migration overhead and image caching, while Nadgowda et al. [6] presented Voyager, unifying CRIU memory checkpointing

TABLE II
Empirical Statistical Distribution Across $N = 36$ Multi-Hop CRIU Live Migrations on AWS EC2

Direction / Subset	T_{freeze} (ms)	T_{build} (ms)	T_{push} (ms)	RPO (ms)	T_{restore} (ms)	T_{blackout} (ms)	$T_{\text{disruption}}$ (ms)	T_{total} (ms)	$c_{\text{kill}} - c_{\text{dump}}$ (ticks)
worker-1 → worker-2 ($n = 20$)									
Median ($p50$) / $p95$	144.0 / 158	368.5 / 400	293.0 / 310	664.0 / 703	1033.0 / 1533	1031.5 / 1532	1178.0 / 1673	1857.0 / 2358	7.5 / 8
Mean ± SD [Min–Max]	143.3 ± 9.4	370.9 ± 17.1	293.6 ± 12.5	664.6 ± 27.6	1112.9 ± 249.2	1111.2 ± 249.3 [817–1687]	1254.5 ± 250.1 [958–1836]	1920.8 ± 248.7	7.6 ± 0.6 [7–9]
worker-2 → worker-1 ($n = 16$)									
Median ($p50$) / $p95$	145.0 / 155	372.5 / 384	301.5 / 308	672.5 / 688	1092.0 / 1474	1090.5 / 1472	1229.0 / 1622	1887.0 / 2277	7.0 / 8
Mean ± SD [Min–Max]	145.4 ± 7.1	366.6 ± 16.4	296.6 ± 12.4	663.2 ± 27.9	1160.2 ± 235.9	1158.8 ± 235.7 [797–1682]	1304.1 ± 238.1 [944–1828]	1968.8 ± 238.0	7.4 ± 0.5 [7–8]
Combined All Runs ($N = 36$)									
Median ($p50$) / $p95$	144.0 / 156	370.5 / 389	298.5 / 310	669.0 / 696	1066.0 / 1533	1064.5 / 1532	1203.0 / 1673	1878.0 / 2358	7.0 / 8
Mean ± SD [Min–Max]	144.2 ± 8.4	369.0 ± 16.7	294.9 ± 12.4	663.9 ± 27.4	1133.9 ± 241.1	1132.4 ± 241.1 [797–1687]	1276.6 ± 242.6 [944–1836]	1942.1 ± 241.8	7.5 ± 0.6 [7–9]

TABLE III
Comparison of Illustrative Traces and Operating Modes Against Native Cold Reschedule

Execution Mode / Trace ID	Path	T_{total}	T_{freeze}	T_{build}	T_{push}	T_{blackout}	$T_{\text{disruption}}$	RPO	Counter($c_{\text{dump}} \to c_{\text{kill}} \to c_{\text{restore}}$)
Native Cold Reschedule (Baseline)	w2 → w1	1289 ms	N/A	N/A	N/A	1289 ms	1289 ms	Total Loss	43544 → 8 (100% RAM Lost)
CRIU Decoupled (Trace #35, Run 1)	w1 → w2	1935 ms	159 ms	385 ms	310 ms	1078 ms	1237 ms (−4.0%)	695 ms	$51 \to 60 \to 57$ ($\Delta S_{\text{lost}} \approx 8, \Delta S_{\text{post}} \approx 5$)
CRIU Decoupled (Trace #36, Run 2)	w2 → w1	1642 ms	147 ms	388 ms	308 ms	797 ms	944 ms (−26.8%)	696 ms	$77 \to 85 \to 79$ ($\Delta S_{\text{lost}} \approx 7, \Delta S_{\text{post}} \approx 1$)
CRIU Median ($N = 36$, Mode A)	Bidirectional	1878 ms	144 ms	370.5 ms	298.5 ms	1064.5 ms	1203 ms (−6.7%)	669 ms	$\Delta S_{\text{lost}} = 7.5 \pm 0.6$ ticks
Stop-and-Copy (Mode B, Analytical)	Bidirectional	1878 ms	144 ms	370.5 ms	298.5 ms	1878 ms	1878 ms (+45.7%)	0 ms	Zero Drift (Blackout = T_{total})

with union filesystem data migration in Docker. Reber [8] integrated forensic container checkpointing into Kubernetes via KEP-2008. Our work extends KEP-2008 into an automated node-to-node migration pipeline over CRI-O and Cilium eBPF [9], quantifying the interplay between CRIU freeze time, OCI registry packaging, and Kubelet PLEG readiness latency.

VI. Threats to Validity and System Limitations

1) Single-Run Cold Baseline & Analytical Mode B: While CRIU migration is evaluated across $N = 36$ runs, the native cold reschedule baseline (1289 ms) was captured as a single reference run, and Mode B (stop-and-copy) is derived analytically from the measured stage components. Multi-run baselines Including multi-replica stateless deployments with PodDisruptionBudget are planned.
2) L4 Connection Closure (tcp-close): Because tcp-close closes active TCP sockets on restore and our workload uses short-lived HTTP/1.0 requests, persistent HTTP/1.1 keep-alive, WebSocket, and UDP sessions were not tested.
3) Single-Pass Memory Scaling: Our empirical runs used a 26 MB container (5.16 MB compressed checkpoint). Validating multi-gigabyte heaps and integrating iterative pre-copy (criu dump –pre-dump) are required for large-memory workloads.

VII. Conclusion

We presented an empirical and analytical evaluation of snapshot-consistent container live migration in Kubernetes using CRI-O, CRIU v4.3, Buildah, and Cilium eBPF. Across $N = 36$ migrations on AWS EC2, the pipeline achieves a median cutover blackout of 1064.5 ms (best 797 ms) and median cumulative disruption of 1203.0 ms (best 944 ms) with a tightly bounded RPO of 669.0 ms (7.5 ± 0.6 ticks), while regression analysis ($r = 0.9624$) isolates Kubelet's 1 s PLEG sync loop as the primary source of post-restore tail latency.

Artifact Availability

All manifests, Ansible playbooks, operator source code, and benchmarking scripts are publicly available at: https://github.com/musaxasmammedov7/final-research.

References

[1] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016.
[2] Netflix Technology Blog, "Creating Your Own EC2 Spot Market," Netflix Inc., 2015.
[3] Amazon Web Services, "Spot Instance Interruptions and EC2 Rebalance Recommendations," AWS Documentation, 2024.
[4] Redis Ltd., "Redis Latency Problems Troubleshooting," https://redis.io/docs/management/optimization/latency/, 2024.
[5] R. Stoyanov and M. J. Kollingbaum, "Efficient Live Migration of Linux Containers," in High Performance Computing: ISC High Performance 2018 International Workshops, LNCS, vol. 11203, Springer, Cham, 2018, pp. 184–193.
[6] S. Nadgowda, S. Suneja, N. Bila, and C. Isci, "Voyager: Complete Container State Migration," in Proc. IEEE 37th Int. Conf. on Distributed Computing Systems (ICDCS), 2017, pp. 2137–2142.
[7] Checkpoint/Restore In Userspace, "CRIU CLI Options and Architecture," https://criu.org, 2024.
[8] A. Reber, "Forensic Container Checkpointing in Kubernetes," Kubernetes Official Blog / CNCF, Dec. 2022. [Online]. Available: https://kubernetes.io/blog/2022/12/05/forensic-container-checkpointing-alpha/
[9] Cilium Authors, "Cilium: eBPF-based Networking, Observability, and Service Mesh," Linux Foundation, 2024.
[10] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live Migration of Virtual Machines," in Proc. 2nd USENIX Symp. on Networked Systems Design and Implementation (NSDI), 2005, pp. 273–286.